

Hands-on Environmental Sensing I

Praktikum - Aufbau und Charakterisierung eines Low-Cost CO₂ Sensors

Daniel Kühbacher

Version: 0.2

Zuletzt bearbeitet: 18. November 2025

Autor: Daniel Kühbacher

Inhalt

Vorbereitung.....	3
Einführung	5
<i>Ablauf.....</i>	<i>5</i>
Aufgabe 1: Verbindung zum Microcontroller	6
Aufgabe 2: Serieller Monitor	5
Aufgabe 3: Temperatur und Luftfeuchtigkeit messen.	6
Aufgabe 4: Inbetriebnahme der SD-Karte	7
Aufgabe 5: Sensordaten auf SD-Karte speichern	8
Aufgabe 6: Inbetriebnahme des CO₂ Sensors	7
Aufgabe 7: CO₂ Beacon	7
<i>Zusatzaufgabe 7.1: Zeitstempel.....</i>	<i>8</i>
Aufgabe 8: Datenauswertung mit Python	9

Vorbereitung

Zur Vorbereitung für das Praktikum sind zwischen 30 min bis 2 h erforderlich, je nach Vorerfahrung mit der **Arduino Microcontroller Programmierung** und **Python**.

Das Praktikum wird in 2er-Gruppen durchgeführt, und jede Gruppe benötigt einen eigenen Laptop. Ihr könnt euch im Voraus absprechen und Gruppen bilden. Falls ihr möchtet, könnt ihr euch einen Laptop teilen und nur einen mitbringen.

- ➔ Installiere vor dem Praktikum die **Arduino IDE**
Link zum Download: <https://www.arduino.cc/en/software>

Zur weiteren Vorbereitung, insbesondere wenn noch keine Erfahrung mit Arduino Programmierung hast, siehe dir das folgende Video an:

Arduino Crash Course in 15 Min

<https://youtu.be/nL34zDTPkcs?si=MqPz2jDbrG39jY->

- ➔ Installiere **Visual Studio Code** und die **Erweiterung für Jupyter Notebooks** (erforderlich für die Datenauswertung in Python)
Link zum Download: <https://code.visualstudio.com/Download>

Zur Einführung und Installation von Jupyter siehe dir das folgende Video an:

<https://youtu.be/suAkMeWJ1yE?si=K90X1-5LGSxWY5A>

- ➔ Lade und entpacke das **GitHub-Repository** zum Kurs auf deinen Laptop: <https://github.com/tum-esm/HES-Praktikum.git>

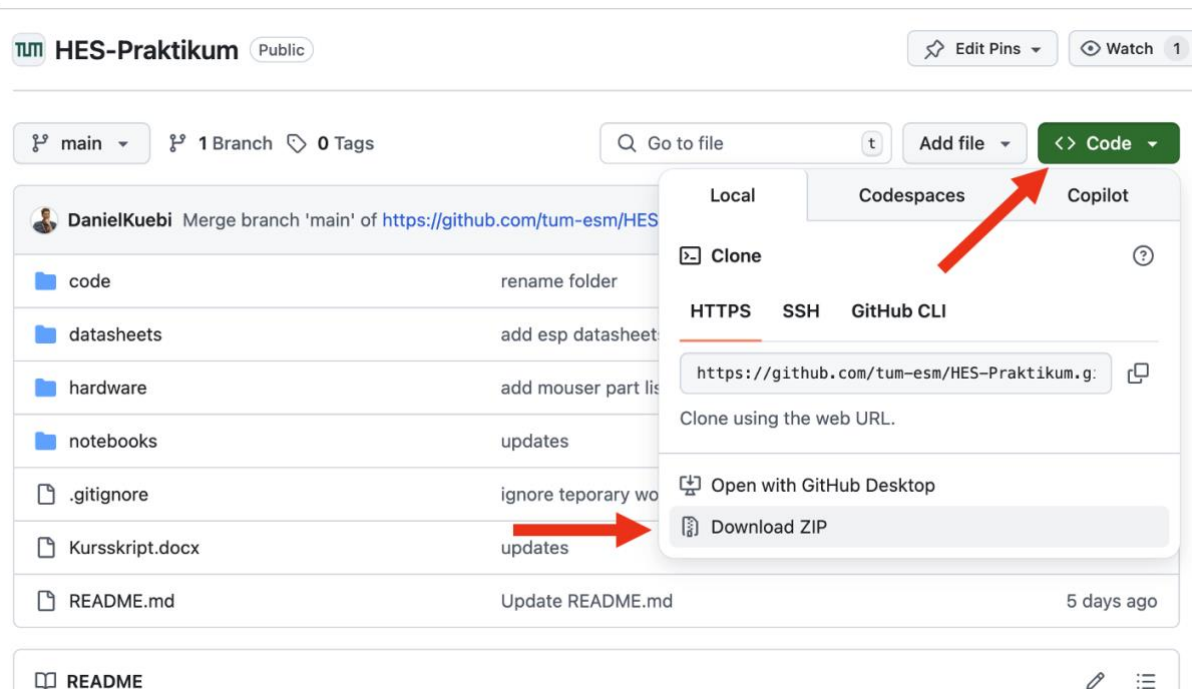


Abbildung 1 - Herunterladen des Github Code Repositories

Einführung

Kohlendioxid (CO_2) macht mit etwa 0,04 % nur einen geringen Anteil der Erdatmosphäre aus. Dennoch hat es eine erhebliche Klimawirkung, da es die von der Erde abgegebene Wärmestrahlung im nahen Infrarotbereich absorbiert und teilweise zurück auf die Erdoberfläche reflektiert – ein Effekt, der als Treibhauseffekt bekannt ist. Ohne diesen natürlichen Treibhauseffekt wäre Leben auf der Erde nicht möglich, da die Durchschnittstemperatur der Erdoberfläche bei etwa -18°C läge. Durch menschliche Aktivitäten, insbesondere die Verbrennung fossiler Brennstoffe, steigt jedoch die CO_2 -Konzentration kontinuierlich an und verstärkt den Treibhauseffekt. Dies führt zu einer globalen Erwärmung mit weitreichenden Folgen für Klima, Wetter und Ökosysteme.

Die präzise Messung von CO_2 ist essenziell, um natürliche und anthropogene Quellen sowie Senken des Treibhausgases besser zu verstehen. Nur so lassen sich gezielte Maßnahmen zur Reduktion von Emissionen entwickeln und die Auswirkungen des Klimawandels effektiver begrenzen. Eine zentrale Methode zur Bestimmung der CO_2 -Konzentration basiert auf dem Lambert-Beer-Gesetz, das die Abschwächung von Lichtintensität durch Absorption beschreibt. CO_2 absorbiert infrarote Strahlung bei spezifischen Wellenlängen, wodurch seine Konzentration in der Luft durch nicht-dispersive Infrarotspektroskopie (NDIR) genau bestimmt werden kann. Diese Methode wird in Laboranalysen, Umweltmessstationen und sogar Satelliten eingesetzt, um den globalen CO_2 -Gehalt kontinuierlich zu überwachen.

Im Praktikum “Hands-On Environmental Sensing” werden wir einen Low-Cost CO_2 Sensor in Betrieb nehmen, Daten erfassen und den Sensor charakterisieren. Dabei wird ein Microcontroller (ESP32-C3) so programmiert, dass er Daten von einem Temperatur- und Luftfeuchtigkeitssensors sowie einem CO_2 Sensor empfangen und an einen Seriellen Monitor senden sowie auf eine SD-Karte speichern kann. Daraufhin wird eine Messkampagne mit Referenzgasen durchgeführt und damit der Sensor charakterisiert.

Ablauf

- (15 min) Einführung in die Zusammensetzung der Atmosphäre, Treibhauseffekt, physikalische Grundlagen und Methoden der CO_2 Gasesstechnik
- (15 min) Inbetriebnahme des Microcontrollers
- (30 min) Inbetriebnahme des Temperatur- & Luftfeuchtigkeitssensors sowie der SD-Karte
- (45 min) Inbetriebnahme des CO_2 Sensors und Implementierung der Messroutine
- (30 min) Unterschiedliche Messungen (Referenzgas, Innenluft, Außenluft)
- (45 min) Auswertung und Visualisierung der Daten in Python

Aufgabe 1: Verbindung zum Microcontroller

Öffne die Arduino IDE. Gehe zu *Tools > Board > Boards Manager* und installiere:

„**esp32 by Espressif Systems**“.

Gehe zu *File > Examples > 01.Basics > Blink*. Ein neues Fenster mit dem Blink Beispiel wird sich öffnen.

ToDo ☒ Verbinde nun den Microcontroller mit dem Computer. Wenn der Microcontroller erkannt wird, kannst du nun den entsprechenden PORT und das verwendete BOARD „*ESP32C3 Dev Module*“ auswählen. Lade das Blink Beispiel hoch und überprüfe, ob die Verbindung zum Microcontroller erfolgreich hergestellt wurde.

Problembehandlung  Sollte der Microcontroller nicht erkannt oder kein PORT in der Arduino IDE angezeigt werden, ist es sehr wahrscheinlich, dass der notwendige Treiber fehlt. Im Ordner „windows treiber“ findest du den richtigen Treiber zur Installation. Entpacke den Ordner und führe die Datei „CP210xVCPInstaller_x64.exe“ aus.

Unter dem folgenden Link findest du Treiber für andere Betriebssysteme zur Installation:

<https://www.silabs.com/developer-tools/usb-to-uart-bridge-vcp-drivers?tab=downloads>

Aufgabe 1.2: RGB LED

Da der ESP32 zusätzlich eine RGB Led verbaut hat, kann die Lichtfarbe verändert werden.

ToDo ☒ Verändere das Blink Beispiel so, dass die LED abwechselnd rot und grün blinkt. Die Variablen können einen Wert von 0-255 annehmen, was der Intensität der Farbe Rot, Blau und Grün entspricht.

```
// Verwendung der RGB LED
// in void setup():
pinMode(RGB_BUILTIN, OUTPUT);
// in void loop():
rgbLedWrite(RGB_BUILTIN, 0, 0, 0); // rot, blau, grün
```

Aufgabe 2: Temperatur und Luftfeuchtigkeit messen.

Verbinde nun den Microcontroller mit dem Temperatur- und Luftfeuchtigkeitssensor AHT20. Die Datenübertragung erfolgt mit I²C, sodass neben der Spannungsversorgung (3V, GND) die Datenleitung SDA (Serial Data) und ein Takt SCL (Serial Clock) verbunden werden müssen. Verbinde **SDA mit GPIO 8** und **SCL mit GPIO 9** des Microcontrollers. Diese sind die Default-Pins für I²C ESP32.

Gehe zu *Sketch > Include Library > Manage Libraries* und installiere „**DFRobot_AHT20 by DFRobot**“.

ToDo ☒ Nachdem die Library erfolgreich installiert wurde, gehe zu *File > Examples > DFRobot_AHT20 > readAHT20*. Ein neuer Sketch mit Beispielcode zum Auslesen des Sensors öffnet sich. Lade den Beispielcode hoch, um die Verbindung zum Sensor zu testen. Öffne zum Anzeigen der Messdaten *Tools > Serial Monitor* und stelle die baud-Rate auf 115200 (Datenübertragungsgeschwindigkeit der Seriellen Verbindung). Dieser Wert kann in der Funktion `Serial.begin(115200)` definiert werden.

Aufgabe 2.1: Temperatur und Luftfeuchtigkeit plotten

Nun wollen wir die Messwerte gerne als Liniengraph am Computer darstellen. Hierzu bietet die Arduino IDE einen Serial Plotter unter *Tools > Serial Plotter*. Dieser erfordert jedoch ein spezielles Tabellenformat, um die Daten zu visualisieren. Mit einem tabspace („\t“) werden die mehrere Variablen in einer Zeile getrennt. Mit dem zeilenumbruch („\n“) wird ein neuer Schritt auf der x-Achse erzeugt.

ToDo ☒ Verändere den Beispielcode so, dass die Temperatur (in °C) und die Luftfeuchtigkeit (in %rH) im Serial Plotter dargestellt werden kann. Überprüfe wie sich der Temperatur- und Luftfeuchtigkeitwert verändert, wenn du z.B.: auf den Sensor atmest.
Speichere den Code zu dieser Aufgabe als neuen Sketch im Code-Ordner des Praktikums.

```
Serial.print(<temperature_value>);
Serial.print("\t"); //tabspace
Serial.print(<humidity_value>);
Serial.print("\n"); //neue Zeile
```

GPIO 8 wird auch für den Betrieb der internen RGB LED verwendet (siehe Abbildung 2), wodurch diese beim Betrieb des Sensors hell leuchtet. Wir können alternativ **GPIO 0 als SDA** und **GPIO 1 als SCL** für I²C definieren. Dazu müssen wir die Verdrahtung entsprechend ändern und im Code ist eine Definition der I²C Pins und eine Re-Initialisierung des I²C Interfaces erforderlich.

```
// Import der Wire Bibliothek und Definition der neuen GPIOs
#include "Wire.h"

#define I2C_SDA 0
#define I2C_SCL 1

// dieser code wird in void setup() durchgeführt
// Initialisierung der I2C Schnittstelle auf custom GPIOs
Wire.begin(I2C_SDA, I2C_SCL);
```

ToDo ☒ Ändere die Verdrahtung der I²C Pins und ändere den Code wie beschrieben, um in weiterer Folge GPIO 0 und GPIO 1 für die I²C Verbindung zu verwenden.

Aufgabe 3: Inbetriebnahme des CO₂ Sensors

Als nächsten Baustein des Systems nehmen wir nun den CO₂ Sensor in Betrieb. Verbinde hierzu den **EN (Enable) Pin mit GPIO 3** und den **RDY (Ready) Pin mit GPIO 2**. Der Sensor wird auch über I²C ausgelesen und kann einfach mit den SCL- und SDA-Pins des Temperatur- und Luftfeuchtigkeitssensors verbunden werden (Bus-Topologie).

Abbildung 2 ist der Microcontroller und die verwendeten Pins dargestellt.

ToDo ☒ Öffne nun den im Repository zur Verfügung gestellten Sketch „co2_sensor.ino“, lade das Skript hoch und überprüfe die Verbindung zum CO₂ Sensor.

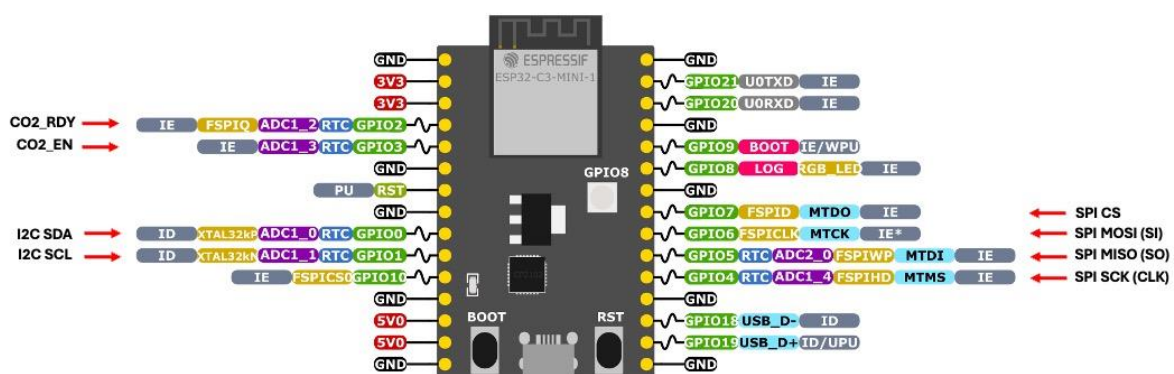


Abbildung 2 – Übersicht über alle GPIOs und ihre Verwendung am ESP32C3 Devkit

Aufgabe 4: Inbetriebnahme der SD-Karte

Verbinde nun die SD-Karte mit dem Microcontroller. Sie benötigt wiederum eine Spannungsversorgung (3V) und wird mittels SPI-Bus (Serial Peripheral Interface) verbunden. Dieser benötigt jedoch im Gegensatz zu I²C zwei Datenleitungen und eine Taktleitung. Verbinden wie folgt **CS (Chip Select) mit GPIO 7**, **SCK (Serial Clock) mit GPIO 4**, **SO/MISO (Master In Slave Out) mit GPIO 5** und **SI/MOSI (Master Out Slave In) mit GPIO 6**.

Öffne den Beispielcode „sd_card.ino“ aus dem *code* Ordner.

Gehe zu *Sketch > Include Library > Manage Libraries* und installiere „**ArduinoJson** by Benoit Blanchon“.

ToDo ☒ Lade den Beispielsketch hoch und überprüfe, ob die Verbindung mit der SD-Karte funktioniert.

Aufgabe 5: Sensordaten auf SD-Karte speichern

Nun haben wir alle Komponenten einzeln in Betrieb genommen und wollen die Sensordaten auf der SD-Karte speichern.

ToDo ☒ Öffne den Beispielcode „co2_beacon.ino“. Dieser Code ist unvollständig und an vielen Stellen mit *//TODO* Kommentaren versehen. Integriere an diesen Stellen jeweils die wesentlichen Code-Bestandteile aus den vorherigen Aufgaben, sodass folgende Funktionen vorhanden sind:

- LED leuchtet rot, während die Setup Routine läuft
- Es werden Luftfeuchtigkeits-, Temperatur- und CO₂-Messungen mit einem bestimmten Messintervall durchgeführt
- Die Daten werden am Seriellen Plotter ausgegeben.
- Die Daten werden auf der SD-Karte gespeichert.

Zusatzaufgabe 5.1: Zeitstempel

Der ESP verfügt über ein WIFI-Modul, wodurch die Möglichkeit besteht, online die aktuelle Zeit abzufragen. Unter *File > Examples > ESP32 > Time > SimpleTime* findest du ein Beispiel dafür, wie diese Zeitabfrage über einen NTP-Server (Network Time Protocol) funktioniert. Öffne den Sketch und gib die SSID und das Passwort für ein WLAN-Netzwerk, beispielsweise vom Handy, an. Nachdem sich das Gerät mit dem Internet verbunden hat, sollte die aktuelle Zeit im Seriellen Monitor dargestellt werden.

ToDo ☒ Integriere die Funktion zum Abrufen von Zeitstempel aus dem Beispielcode in Aufgabe 7. Die Zeitstempel sollen lediglich auf der SD-Karte zu jedem Datenpaket gespeichert werden, nicht aber im seriellen monitor angezeigt. Zur Hilfestellung befinden sich nachfolgend noch einige Code-Ausschnitte.

```
//config the time
configTime(0, 0, ntpServer1, ntpServer2); //Zeitzone ist UTC, kein timeshift

// timestamp erzeugen
struct tm timeinfo;
char timestamp[30];
if (!getLocalTime(&timeinfo)) {
    Serial.println("No time available (yet)");
    return;
}

//String Formatierung für timestamp
strftime(timestamp, sizeof(timestamp), "%Y-%m-%dT%H:%M:%SZ", &timeinfo);

// Werte in JSON variable schreiben
measurementData["timestamp"] = timestamp;
```


Aufgabe 6: Datenauswertung mit Python

Die erhobenen Daten können nun eingelesen, geplottet und weiterverarbeitet werden. Hierzu nutzen wir Python und ein Jupyter Notebook. Lege nun die SD-Karte in den Computer ein und kopiere das Datenfile in den Ordner *notebooks/data*.

Öffne mit Hilfe von Visual Studio Code den Ordner *notebooks* und öffne die Datei *analysis_notebook.ipynb* aus diesem Ordner.

Im *notebook/data* Ordner befindet sich außerdem ein über mehrere Tage erhobenes Datenfile des Sensors. Solltest du nicht selbst ausreichend Daten gesammelt haben kannst du das Notebook mit den verfügbaren Daten durchlaufen lassen.

ToDo  Die weiteren Schritte zur Datenanalyse sind im analysis notebook beschrieben. Führe die Aufgaben dort aus.

Prüfungsfragen

Welches physikalische Gesetz wird bei der optischen Messung von Spurengasen in der Atmosphäre angewendet? Beschreibe dieses kurz.

Lösung: Beer-Lambert Gesetz; Bei atmosphärischen Messungen ermöglicht dieses Gesetz die Berechnung der Konzentration von Spurengasen anhand der gemessenen Lichtabschwächung bei charakteristischen Wellenlängen.

Nenne mindestens 3 Bus-Protokolle, die typischerweise zur Kommunikation zwischen Mikrocontrollern und Peripheriegeräten eingesetzt werden (Abkürzung ist ausreichend).

Lösung:

- SPI (Serial Peripheral Interface)
- I2C (Inter-Integrated Circuit)
- UART (Universal Asynchronous Receiver Transceiver)
- CAN (Controller Area Network)

Nenne 3 Herausforderungen von Low-Cost CO₂ Sensoren im Vergleich zu wesentlich teureren Gas-Analysatoren (z.B.: Picarro G2401).

Lösung:

- Geringere Messgenauigkeit → Höhere Streuung in den Messwerten (Rauschen)
- Zeitlicher Drift des Mess-Signals → Regelmäßige Neukalibrierung erforderlich
- Empfindlichkeit gegen Umwelteinflüsse → Temperatur und Luftfeuchtigkeit haben oft einen großen Einfluss auf das gemessene Signal und müssen mitbetrachtet werden.

Beschreibe kurze den Nutzen und die Unterschiede der Funktionen *void setup()* und *void loop()* bei der Arduino Microcontroller Programmierung?

Lösung: Die Funktion *void setup()* wird nur einmal zum Beginn des Programmablaufs durchgeführt. Sie dient zur Initialisierung von z.B.: Pin Modi, der seriellen Kommunikation oder zur Initialisierung von Sensoren. In der Funktion *void loop()* befindet sich das Hauptprogramm und sie wird nach Abschluss der Setup-Routine endlos wiederholt.